Examen 2 — Arquitectura Cloud Segura (ReMarket)

Criptografía y Seguridad 2027-1 · Facultad de Ciencias, UNAM
Profesora: María Ximena | Noviembre 2026 | Equipo de 3 integrantes

Integrante	Rol Principal	Usuario GitHub
Oscar David Hernández Rodríguez	Criptografía en tránsito (Bloque B)	**OscarHeRo**
Ian Alejandro Arroyo Ramírez	Arquitectura y datos (Bloque A)	**IanFoi**
Violeta Ardeni Castillo Camacho	Datos en reposo, riesgos y pipeline (Bloque C)	**chateau25**

Resumen Ejecutivo

ReMarket es un e-commerce en microservicios sobre AWS diseñado bajo **Zero Trust**. La arquitectura define 6 zonas de confianza, mTLS interno con identidades SPIFFE emitidas por Istio, cifrado en reposo AES-256 con *envelope encryption* en AWS KMS y contraseñas derivadas con Argon2id. Los cobros se delegan a Stripe bajo PCI-DSS SAQ-A sin almacenar datos de tarjeta, los comprobantes se firman con ECDSA P-256 y los webhooks se validan con HMAC-SHA256, ventana de tiempo y registro de eventos ya procesados. El pipeline DevSecOps integra escaneos SAST/SCA bloqueantes y firmas Cosign. La infraestructura es Multi-AZ (RTO ≤ 5 min y RPO = 0 en pagos ante la caída de una zona), y el documento cierra con una evaluación crítica del uso de IA en ciberseguridad.

1. Arquitectura General del Sistema

ReMarket opera sobre **AWS EKS** (Kubernetes) con **Istio Service Mesh**, estructurada en **6 Zonas de Confianza**:

- **Z0 (No Confiable):** Navegadores de clientes y equipos del personal interno.
- **Z1 (Perimetral - DMZ):** CloudFront, AWS WAF (filtrado L7) y AWS Shield (DDoS L3/L4), ALB, Istio Ingress Gateway, SPA en S3 y VPN administrativa.
- **Z2 (Aplicación EKS):** Microservicios privados (Auth, Catálogo, Pedidos, Inventario, Pagos, Notificaciones), Redis y el bus SNS → SQS.
- **Z3 (Persistencia Aislada):** Subredes sin salida a Internet: RDS PostgreSQL de usuarios, RDS transaccional Multi-AZ, DynamoDB y bitácoras en S3.
- **Z4 (Gestión y Seguridad):** AWS KMS, CloudHSM, Secrets Manager, IAM (IRSA), CA de Istio y SIEM.
- **Z5 (Servicios Externos):** Proveedores vía HTTPS: Stripe (pagos SAQ-A), paqueterías (DHL/FedEx) y mensajería (Twilio/SES).

Componente	Implementación	Zona	Función y Control de Seguridad
Frontend web	SPA en S3 servida por CloudFront	Z1	Distribución estática con CSP y HSTS Preload.
API Gateway	Istio Ingress Gateway (Envoy) tras ALB	Z1	Entrada única, validación JWT/OpenAPI y rate limiting.
Auth y Usuarios	Pods en EKS privado	Z2	Emisión de JWT (ES256) y contraseñas con Argon2id.
Catálogo	Pods en EKS privado	Z2	Lecturas desacopladas de persistencia transaccional.
Pedidos	Pods en EKS privado	Z2	Orquestación, idempotencia en Redis y firma de comprobantes.
Inventario	Pods en EKS privado	Z2	Reserva atómica de existencias contra *overselling*.
Pagos	Pods en EKS privado	Z2	Coordinación con Stripe y validación HMAC de webhooks.
Notificaciones	Pods en EKS privado	Z2	Consumo asíncrono y envío de correos/SMS.
BD Usuarios	RDS PostgreSQL	Z3	Datos personales y hashes; sólo accesible por Auth.
BD Transaccional	RDS PostgreSQL Multi-AZ	Z3	Un esquema y usuario de BD por servicio; réplica síncrona.

Componente	Implementación	Zona	Función y Control de Seguridad
BD Catálogo	Amazon DynamoDB + S3	Z3	NoSQL para lecturas con cifrado KMS transparente.
Caché	ElastiCache Redis	Z2	Sesiones de refresco y llaves de idempotencia (TTL 24 h).
Bus de eventos	Amazon SNS → SQS	Z2	Fan-out a una cola por consumidor, permisos IAM y firma ECDSA.
Ext. Pagos	Stripe API	Z5	Tokenización; ReMarket no toca tarjetas (PCI SAQ-A).
Ext. Paquetería	APIs DHL / FedEx	Z5	Envíos vía TLS autenticados con tokens API rotados.

Aislamiento de Red

La segmentación en zonas impide el movimiento lateral: los pods no tienen IP pública y todo lo que entra (incluido el webhook de Stripe) pasa por CloudFront, WAF, ALB y el Ingress Gateway. Si un servicio expuesto como Catálogo es comprometido, las reglas de red y las políticas IAM impiden cualquier conexión hacia las bases de datos de usuarios o pagos (detalle en la sección 13). El diagrama completo está en el **Anexo A** al final del documento.

2. Clasificación de Información

Clasificamos los activos en cuatro niveles (**Crítica**, **Confidencial**, **Interna** y **Pública**). El nivel decide el control: lo crítico se cifra, se aísla y se audita; lo público sólo necesita integridad y disponibilidad.

Tipo de Dato	Nivel	1. Generación	2. Almacenamiento	3. Circulación	4. Acceso Autorizado
1. Credenciales	Crítica	Navegador (registro y login)	BD Usuarios (sólo hash Argon2id)	Frontend → Gateway → Auth	Sólo servicio Auth
2. Datos Personales	Confidencial	Cliente (registro)	BD Usuarios (AES-256)	Gateway → Auth / Pedidos	Cliente, Auth y soporte
3. Direcciones	Confidencial	Cliente (checkout)	BD Transaccional (AES-256)	Pedidos → Paquetería	Pedidos y paquetería
4. Pedidos	Confidencial	Servicio Pedidos	BD Transaccional (esquema pedidos)	Pedidos → Pagos → bus	Cliente, Pedidos, Notificaciones
5. Inventario	Interna	Inventario / ERP	BD Transaccional y Redis	Inventario → Catálogo, Pedidos	Inventario (escritura); el público sólo ve “disponible / agotado”
6. Pagos	Crítica	Stripe Elements	Sólo ID de pago y estado; nunca tarjeta	Cliente → Stripe → Pagos	Sólo servicio Pagos
7. Tokens de Sesión	Crítica	Servicio Auth	Acceso: memoria del navegador; refresco: cookie HttpOnly + Redis	Cliente → Gateway → Pods	Gateway y microservicios
8. Bitácoras de Auditoría	Confidencial	Envoy y microservicios	S3 Object Lock (WORM)	Microservicios → SIEM	Equipo de seguridad
9. Claves Criptográficas	Crítica	AWS KMS / CloudHSM	KEKs dentro del HSM (no exportables)	Sólo DEKs, en memoria del pod	Roles IAM (IRSA)

5–7. Impacto si se Compromete Confidencialidad, Integridad o Disponibilidad

Tipo	(C) Si se filtra	(I) Si se modifica	(D) Si no está disponible
1. Credenciales	Robo de cuentas y *credential stuffing*	Suplantación o bloqueo del titular	Nadie puede iniciar sesión
2. Datos personales	Sanción LFPDPPP/GDPR y pérdida de confianza	Datos de contacto falsos, fraude	Sin perfil ni historial
3. Direcciones	Riesgo físico para el cliente	Paquete enviado a un tercero	No se puede entregar

Tipo	(C) Si se filtra	(I) Si se modifica	(D) Si no está disponible
4. Pedidos	Exposición de hábitos de compra	Cambio de productos o montos	No se pueden crear ni consultar órdenes
5. Inventario	Ventaja a competidores	Sobreventa o productos bloqueados	Ventas detenidas
6. Pagos	Fraude financiero	Pedido marcado como pagado sin cobro	Caja detenida
7. Tokens de sesión	Secuestro de sesión	Escalada de privilegios	Usuarios expulsados
8. Bitácoras	Exposición de topología interna	Borrado de huellas forenses	Ceguera ante incidentes
9. Claves	Descifrado masivo y firmas falsas	Suplantación integral del sistema	Datos cifrados inaccesibles para la propia empresa

3. Protección de la Página Web (Usuario ↔ ReMarket)

Qué problema resuelve HTTPS: sin él, cualquiera en la misma red (Wi-Fi pública, ISP, proxy) podría leer contraseñas y tokens, modificar la página o los montos en tránsito, o hacerse pasar por la tienda. HTTPS es HTTP dentro de **TLS**, que da tres garantías: autenticación del servidor, confidencialidad e integridad.

La conexión termina en CloudFront (y CloudFront re-cifra hacia el ALB). Usamos **TLS 1.3** como preferente y aceptamos **TLS 1.2 sólo con ECDHE y suites AEAD** (política `TLSv1.2_2021`): bloquear TLS 1.2 por completo dejaría fuera a clientes internacionales con equipos antiguos, y con esas suites TLS 1.2 no tiene debilidades prácticas conocidas. SSLv3, TLS 1.0 y 1.1 quedan bloqueados para evitar ataques de degradación.

Configuración y Garantías Criptográficas

1. **Autenticación del servidor (certificados X.509):** ACM emite el certificado de `remarket.com` con clave **ECDSA P-256**, firmado por una CA reconocida. El navegador verifica la cadena hasta una CA de confianza, que el nombre coincida y la vigencia; luego el servidor firma el *handshake* con su clave privada, demostrando que la posee. Un atacante *Man-in-the-Middle* no puede presentar un certificado válido para el dominio.
2. **Confidencialidad e integridad (AEAD):** Cada registro TLS se cifra y autentica con `TLS_AES_256_GCM_SHA384` o `TLS_CHACHA20_POLY1305_SHA256` (móviles sin aceleración AES). Un byte alterado en tránsito hace fallar la etiqueta de autenticación y la conexión se corta.
3. **Secreto hacia adelante (PFS):** Intercambio efímero **ECDHE sobre X25519**. Si la clave privada del servidor se filtra después, las sesiones capturadas antes no pueden descifrarse.
4. **HSTS Preload:** `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload` obliga al navegador a usar sólo HTTPS y evita la degradación a texto claro (*SSL stripping*).

Se excluyen suites vulnerables (CBC, RC4, intercambio RSA estático), y TLS 1.3 reduce el *handshake* a un solo viaje de ida y vuelta (1-RTT).

4. Comunicación entre Microservicios

Bajo **Zero Trust**, la red interna de Kubernetes no se considera segura. El tráfico este-oeste pasa por **Istio Service Mesh** con proxies **Envoy**; la autenticación mutua se aplica según el tramo:

Tramo de Comunicación	Mecanismo	¿mTLS?	Justificación Técnica
Gateway → Microservicios	mTLS estricto + JWT	**SÍ**	El Ingress Gateway es parte del mesh: el servicio comprueba que la llamada venga del Gateway y no de otro pod.
Microservicio → Microservicio	mTLS estricto + JWT	**SÍ**	Evita suplantaciones y espionaje este-oeste si un pod es comprometido.

Tramo de Comunicación	Mecanismo	¿mTLS?	Justificación Técnica
Microservicio → Redis	TLS + token AUTH por servicio	**NO**	ElastiCache no admite certificados de cliente: el servidor se autentica por TLS y el cliente con su token (rotado en Secrets Manager).
Microservicio → RDS	TLS + autenticación IAM	**NO**	RDS se autentica por TLS y el servicio con un token IAM temporal (IRSA, 15 min), sin contraseña fija.
Microservicio → SNS/SQS y KMS	TLS + firma SigV4	**NO**	Servicios gestionados de AWS: cada petición va firmada con credenciales temporales del rol del servicio.

En los tres últimos tramos **ambas partes sí se autentican**, pero el cliente lo hace con un token o una firma, no con un certificado.

Identidad SPIFFE y Rotación

Cada pod recibe una identidad **SPIFFE** (`spiffe://remarket.internal/ns/prod/sa/pedidos`). La CA de Istio (*istiod*) emite certificados X.509 de **24 horas**, que Envoy rota en memoria sin reiniciar contenedores.

En capa 4, Envoy autentica al servicio emisor por mTLS; en capa 7, el microservicio valida el JWT para autorizar al usuario final.

5. API REST — Caso de Estudio `POST /pedido`

`POST /pedido` es el endpoint más crítico del flujo de compra: crea la orden, reserva inventario e inicia el cobro.

```
POST /pedido HTTP/1.1
Host: api.remarket.com
Authorization: Bearer eyJhbGciOiJFUzI1NiIsInR5cCI6IkpXVCJ9...
Idempotency-Key: 9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d
Content-Type: application/json

{"items": [{"sku": "SKU-102", "cantidad": 1}], "direccion_id": "dir_31", "payment_method": "pm_1N2eZ4LkdIwHu7ix"}
```

Controles de Seguridad en la Petición

1. **Autenticación:** JWT de acceso firmado con **ECDSA P-256 (`ES256`)**, vigencia de 15 min. El Gateway verifica firma, `exp`, `aud` e `iss`. El token vive sólo en memoria de la SPA; el token de refresco va en una cookie `HttpOnly; Secure; SameSite=Strict`, inaccesible para JavaScript ante un XSS.
2. **Autorización:** El cliente se toma del `sub` del token, nunca del cuerpo, y `direccion_id` debe pertenecer a ese `sub`. Así nadie crea pedidos a nombre de otro ni usa direcciones ajenas (falla **BOLA**).
3. **Protección contra repetición (*replay*):** Cabecera `Idempotency-Key` (UUIDv4) guardada en **Redis** durante **24 h**, más que la vida del token. Un reenvío, por error de red o malicioso, devuelve la respuesta original sin duplicar cobro ni orden.
4. **Confidencialidad e integridad:** Todo viaja cifrado con TLS (sección 3) y mTLS dentro del clúster (sección 4).
5. **Minimización de datos:** El cuerpo **no contiene tarjeta, CVV ni vencimiento**, sólo el identificador opaco que emite Stripe Elements (PCI-DSS SAQ-A). Tampoco viaja **precio ni total**: el servidor los calcula desde el catálogo, así que el cliente no puede alterarlos. La dirección se referencia por ID, sin reenviar los datos personales.

6. Protección de Datos Almacenados (Datos en Reposo)

Por qué no basta HTTPS: TLS protege el *canal* mientras los datos viajan; al llegar a su destino se descifran y se guardan. Si un atacante roba un disco, un *snapshot* o un respaldo, o entra a la base de datos con credenciales robadas, TLS ya no interviene. Por eso los datos se cifran también en reposo, con claves guardadas lejos de los datos.

Cifrado de Sobre (*Envelope Encryption*)

Cifrar millones de registros llamando al HSM por cada uno sería lento, así que usamos *envelope encryption* con **AES-256-GCM** y **AWS KMS** (HSM validados FIPS 140 nivel 3):

1. **Clave maestra (KEK):** Vive dentro de KMS y **nunca sale del HSM en claro**. Hay una por servicio y tipo de dato.
2. **Claves de datos (DEK):** KMS genera una DEK; el servicio la usa en memoria para cifrar y autenticar el campo o archivo con AES-256-GCM y descarta la copia en claro. Junto al dato sólo se guarda la DEK cifrada por la KEK. Para leer, el servicio pide a KMS descifrar la DEK, lo que exige un rol IAM autorizado y queda en CloudTrail.
3. **Cifrado de discos y almacenamiento:** Volúmenes EBS, RDS, DynamoDB, S3 y snapshots usan cifrado nativo con claves KMS propias (no las administradas por defecto), así que sin permiso de KMS un snapshot copiado es ilegible.
4. **Cifrado de respaldos:** Los respaldos se cifran con una **clave KMS distinta** y se copian a una **cuenta de AWS separada** con S3 Object Lock en modo cumplimiento (WORM), para que ni credenciales robadas de producción ni un *ransomware* puedan borrarlos o leerlos.
5. **Separación de claves, datos y secretos:** Las claves están en KMS, los datos en RDS/S3 y las contraseñas de servicios en Secrets Manager; cada uno con permisos IAM distintos. Robar un solo componente no basta.

Límite honesto: el cifrado en reposo no detiene una inyección SQL, porque la aplicación descifra al consultar. Contra eso aplican consultas parametrizadas, el usuario de BD de mínimo privilegio por servicio y el WAF.

7. Gestión de Contraseñas

El servicio de autenticación sólo necesita comprobar que la contraseña escrita coincide con la registrada, nunca recuperarla. Por eso **no se guarda cifrada de forma reversible**: si alguien obtiene la clave de descifrado (o el código que la usa), recupera las contraseñas de todos los clientes y puede reutilizarlas en otros sitios (*credential stuffing*).

Concepto	Qué es	¿Sirve para contraseñas?
Cifrado	Transformación reversible con una clave (AES, RSA)	No: quien tenga la clave recupera todo
Hash	Función de un solo sentido y rápida (SHA-256)	Sola no: una GPU prueba miles de millones por segundo
Salt	Valor aleatorio único por usuario que se mezcla con la contraseña	Necesario: contraseñas iguales dan resultados distintos e inutiliza tablas precalculadas
KDF de contraseña	Hash deliberadamente lento y costoso en memoria (Argon2id, scrypt, bcrypt)	Sí: encarece cada intento de fuerza bruta

Derivación con Argon2id

Usamos **Argon2id** (RFC 9106), ganador de la *Password Hashing Competition*:

1. **Salt:** 16 bytes de un CSPRNG por credencial, guardados junto al resultado.
2. **Parámetros:** memoria $m = 64$ MiB, $t = 3$ iteraciones y $p = 4$ hilos (una de las configuraciones que recomienda el RFC 9106). Se guardan con el hash para poder subirlos en el futuro y re-derivar al siguiente inicio de sesión.

Exigir 64 MiB por intento hace que un atacante con la base robada sólo pueda probar unas pocas contraseñas por segundo en cada GPU, en lugar de miles de millones con SHA-256. No lo imposibilita, pero lo vuelve impráctico para contraseñas no triviales. Como el costo también lo paga nuestro servidor, `/login` tiene rate limiting por IP y por cuenta en el WAF, para que nadie lo use como vector de denegación de servicio.

8. Gestión de Claves Criptográficas

Por qué las claves no van en el código fuente: el código se copia a cada laptop, al sistema de CI y al historial de Git, donde un secreto queda para siempre aunque luego se borre. Cualquiera con acceso de lectura al repositorio tendría la clave, y cambiarla exigiría recompilar y redesplegar en plena emergencia.

Por eso las claves viven en **AWS KMS** (KEKs y firma de eventos), **AWS CloudHSM** (llave privada de firma de comprobantes) y **AWS Secrets Manager** (contraseñas de servicios, token de Redis, *webhook secret* de Stripe). Los

microservicios las obtienen en ejecución con su rol IAM; en el caso de KMS y CloudHSM, la clave nunca llega al pod: el pod pide la operación.

Ciclo de Vida de las Claves

1. **Generación:** Dentro del HSM con su generador de números aleatorios certificado.
2. **Almacenamiento:** Las KEKs y llaves de firma son **no exportables**; sólo salen DEKs cifradas.
3. **Acceso:** Políticas IAM de mínimo privilegio por servicio (IRSA). Cada uso queda en **CloudTrail**.
4. **Rotación:** KMS rota la KEK cada año (las versiones viejas siguen descifrando datos antiguos); Secrets Manager rota secretos de servicios cada 90 días; Istio rota los certificados mTLS cada 24 h.
5. **Revocación:** Una clave comprometida se deshabilita (*Key State: Disabled*) y deja de servir de inmediato; un secreto filtrado se rota y la versión anterior se invalida.
6. **Respaldo:** KMS replica sus claves entre zonas internamente; el clúster CloudHSM tiene HSM en varias zonas con respaldos cifrados.
7. **Eliminación:** KMS impone una espera de 7 a 30 días antes de borrar. Al borrarse la KEK, todo lo cifrado con ella queda irrecuperable (*borrado criptográfico*), útil para retirar datos de forma verificable.

9. Pasarela de Pagos

El procesamiento de cobros se delega a **Stripe** con **Stripe Elements**, lo que deja a ReMarket en el alcance mínimo de PCI-DSS (**SAQ-A**).

Flujo Usuario → ReMarket → Pasarela

1. **Lo que maneja directamente la pasarela:** El cliente escribe número de tarjeta (PAN), CVV y vencimiento dentro de un *iframe* servido por Stripe; esos datos van del navegador a Stripe sin pasar por nuestros servidores.
2. **Lo que recibe ReMarket:** Un identificador opaco del método de pago (`pm_...`) y, tras el cobro, el ID del `PaymentIntent`, su estado y los últimos 4 dígitos y marca para mostrarlos al cliente.
3. **Lo que nunca se almacena:** PAN completo, CVV (tampoco lo guarda Stripe tras autorizar), banda magnética ni PIN.
4. **Comunicaciones a proteger:** navegador ↔ Stripe (TLS, lo controla Stripe); Pagos → API de Stripe (TLS + llave secreta restringida guardada en Secrets Manager); Stripe → nuestro webhook (TLS + firma HMAC).
5. **Protección del checkout:** Contra robo de datos en el navegador (*Magecart*), CSP estricta (`script-src 'self' https://js.stripe.com; frame-src https://js.stripe.com; connect-src 'self' https://api.stripe.com`) y SRI en **nuestros** scripts. Stripe.js no admite SRI porque se actualiza; se carga siempre desde `js.stripe.com`.

Verificación del Webhook `POST /payment-confirmed`

Para impedir que un atacante envíe un "pago aprobado" falso y la tienda despache sin cobrar, el servicio de Pagos aplica cuatro verificaciones antes de confirmar:

1. **Firma HMAC-SHA256:** Stripe envía en `Stripe-Signature` la marca de tiempo `t` y la firma $\text{HMAC-SHA256}(\text{secreto}, t \parallel \text{"."} \parallel \text{cuerpo})$. El servicio la recalcula con el secreto guardado en Secrets Manager y compara en **tiempo constante**. Sin el secreto, el atacante no puede producir una firma válida.
2. **Ventana de tiempo:** Se rechaza si `t` difiere más de 300 s del reloj del servidor (NTP), así no se puede reenviar una notificación vieja capturada.
3. **Idempotencia por evento:** El `event.id` procesado se guarda; un reenvío dentro de la ventana se ignora.
4. **Confirmación con el proveedor:** Antes de despachar, Pagos consulta `GET /v1/payment_intents/{id}` y comprueba que el estado sea `succeeded` y que **monto y moneda coincidan con el pedido**. Sólo entonces publica `PedidoPagado`.

10. Firmas Digitales e Integridad (Facturas y Comprobantes)

Los comprobantes de compra deben garantizar **integridad** (nadie cambió montos ni productos), **autenticidad** (los emitió ReMarket) y **no repudio** (ReMarket no puede negar haberlos emitido).

Firmar no es cifrar. Cifrar oculta el contenido para que sólo quien tiene la clave lo lea (confidencialidad). Firmar deja el documento **legible para todos** y le agrega una prueba, hecha con la clave privada del emisor, que cualquiera verifica con la clave pública. Un comprobante firmado se puede leer libremente; lo que no se puede es alterarlo sin que la verificación falle.

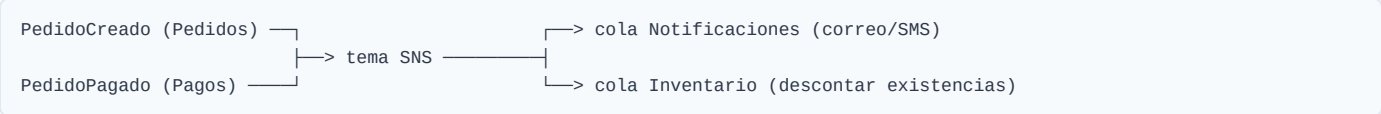
Flujo: Documento → Hash → Firma → Verificación

1. **Documento (M):** El servicio de Pedidos genera el comprobante (XML o PDF).
2. **Hash:** $h = \text{SHA-256}(M)$. Cambiar un solo carácter produce un resumen completamente distinto.
3. **Firma:** La llave privada de ReMarket, custodiada **no exportable en CloudHSM**, firma el resumen con **ECDSA P-256**: $\sigma = \text{Sign}_{sk}(h)$. Pedidos pide la operación; la llave nunca llega a su memoria.
4. **Certificado:** Se adjuntan σ y el certificado **X.509** de ReMarket, emitido por una CA, que liga la clave pública pk con la identidad de la empresa.
5. **Verificación:** Cualquiera (cliente, auditor, paquetería) recalcula $\text{SHA-256}(M)$ y comprueba $\text{Verify}_{pk}(h, \sigma)$. Si el documento cambió, el hash cambia y la firma no valida.

Facturas fiscales en México: el CFDI 4.0 sigue el esquema del SAT: se sella con el Certificado de Sello Digital (CSD) del emisor (RSA con SHA-256) y luego lo timbra un PAC. El mismo flujo hash → firma → verificación aplica; sólo cambian el algoritmo y la CA (el SAT). La llave del CSD también se guarda en CloudHSM.

11. Comunicación Asíncrona (Cola de Eventos)

Para no hacer esperar al cliente, lo que ocurre después de crear o pagar un pedido se procesa en diferido con **Amazon SNS → SQS**: Pedidos y Pagos publican en un tema SNS, y cada consumidor tiene su propia cola SQS suscrita (*fan-out*), así un mismo evento llega a todos.



Controles de Seguridad en la Mensajería

1. **Autenticación del productor y del consumidor:** Cada servicio usa credenciales temporales de su rol IAM (IRSA), y cada petición a SNS/SQS va firmada con **SigV4**. No hay contraseñas compartidas.
2. **Protección del canal:** El tráfico va por **VPC Endpoints** privados con TLS; las políticas de tema y cola exigen la condición `aws:SourceVpce`, así que no se puede publicar desde Internet aunque se tengan credenciales.
3. **Permisos mínimos:** Sólo el rol de Pedidos puede publicar `PedidoCreado` y sólo el de Pagos `PedidoPagado` (`sns:Publish`). Notificaciones e Inventario sólo tienen `sqs:ReceiveMessage` y `sqs:DeleteMessage` sobre **su** cola; ningún consumidor puede publicar.
4. **Integridad del evento:** Temas y colas se cifran con **SSE-KMS**. Además, `PedidoPagado` lleva una firma **ECDSA P-256** hecha con una clave asimétrica de KMS que sólo Pagos puede usar; Inventario y Notificaciones la verifican con la clave pública antes de actuar.
5. **Eventos repetidos o fallidos:** Cada evento lleva un ID único y los consumidores ignoran los ya procesados (idempotencia). Los mensajes que fallan varias veces van a una cola de mensajes fallidos (**DLQ**) con alarma.

¿Qué pasaría si un atacante publicara mensajes falsos?

Un `PedidoPagado` falso haría que el almacén enviara mercancía no pagada; compras o cancelaciones inventadas desfasarían el inventario (sobreventa), y miles de eventos falsos agotarían el presupuesto de SMS/correo y llevarían el dominio a listas negras de spam.

Para lograrlo el atacante necesitaría el rol IAM del productor, estar dentro de la VPC **y**, en el caso de `PedidoPagado`, usar la clave de firma de KMS; tres barreras independientes.

12. Pipeline DevSecOps y Cadena de Suministro

ReMarket aplica *Shift-Left Security*. Flujo: Desarrollador → Git (PR) → Build y pruebas → SAST/SCA → Imagen → Staging → Aprobación → Producción. Cada control va en la etapa más temprana donde detecta el problema, y los críticos bloquean el avance.

Control	Etapa	Herramienta	¿Bloquea?	Propósito Técnico
Detección de secretos	Pre-commit y CI	Gitleaks / TruffleHog	**SÍ**	El hook local se puede saltar; el escaneo en CI no.
Control de acceso	Repositorio	Branch protection, CODEOWNERS, MFA	**SÍ**	Nadie hace push directo a `main` ni cambia el pipeline sin el equipo de seguridad.
Revisión de código	Pull Request	Reglas de GitHub	**SÍ**	Dos aprobaciones antes del merge a `main`.
Pruebas automatizadas	CI	Pruebas unitarias e integración	**SÍ**	Una prueba fallida detiene el build.
Análisis SAST	CI	SonarQube / Semgrep	**SÍ**	Detecta inyección SQL, XSS y mal uso de criptografía.
Análisis SCA	CI	Snyk / Trivy	**SÍ**	Bloquea dependencias con vulnerabilidades CVSS ≥ 7.0 con parche.
Escaneo de imagen	Empaquetado	Trivy	**SÍ**	Revisa paquetes del sistema base antes de subir a ECR.
Firma de imagen	Empaquetado	Sigstore Cosign	**SÍ**	Kyverno en EKS rechaza imágenes sin firma válida.
Pruebas DAST	Staging	OWASP ZAP	**Sólo críticos**	Escaneo dinámico; alerta salvo hallazgos críticos confirmados.
Aprobación a producción	Antes de producción	Entornos de GitHub + ArgoCD	**SÍ**	Visto bueno del líder técnico y de seguridad, con MFA.

Manejo Seguro de Secretos

El pipeline **no guarda claves fijas de AWS**: GitHub Actions usa federación **OIDC** con IAM para obtener credenciales de 15 minutos, limitadas a publicar en Amazon ECR. Los secretos de la aplicación nunca pasan por el pipeline: los pods los leen en ejecución desde Secrets Manager.

Seguridad de la Cadena de Suministro

SCA frena librerías vulnerables o falsificadas (*typosquatting*), Kyverno sólo admite imágenes firmadas por nuestro pipeline y todo cambio productivo exige commits firmados y revisados por dos personas.

13. Seguridad de Contenedores y Microservicios

El objetivo es que vulnerar un microservicio no permita escalar al host ni moverse a otros servicios.

Endurecimiento de Contenedores (*Hardening*)

1. **Imágenes Base Mínimas (Distroless):** Sin gestor de paquetes, `curl`, `wget` ni shell: el atacante no encuentra herramientas tras el compromiso inicial.
2. **Ejecución sin Privilegios (*Rootless*):** Configurada mediante el `securityContext` del pod: `runAsNonRoot: true`, `allowPrivilegeEscalation: false`, sistema de archivos en solo lectura (`readOnlyRootFilesystem: true` con almacenamiento efímero en `emptyDir` RAM) y remoción total de capacidades del kernel (`capabilities: drop: ["ALL"]`).
3. **Actualización de Imágenes:** Las imágenes se reconstruyen cada semana sobre la imagen base más reciente y ECR vuelve a escanear las ya publicadas; una CVE crítica nueva dispara la reconstrucción y el despliegue sin esperar al siguiente cambio de código.

4. **Cuentas de Servicio y Secretos:** Cada microservicio tiene su propia *ServiceAccount* ligada a un rol IAM (IRSA) con sólo los permisos que usa; los secretos se montan en memoria (`tmpfs`) desde Secrets Manager, nunca como variables en la imagen.
5. **Políticas de Red por Defecto Restrictivas:** *NetworkPolicies* de Kubernetes en modo *default-deny* que descartan todo tráfico intra-clúster que no haya sido declarado explícitamente entre pares de pods.

Contención de Brechas y Prevención de Movimiento Lateral

Para verificar la efectividad de la contención en capas, consideramos el microservicio de **Catálogo**, que por atender millones de consultas públicas presenta la mayor exposición perimetral. Ante un escenario hipotético de ejecución remota de código (RCE) en dicho pod, **tres barreras independientes impiden el acceso a las bases de datos de usuarios o pagos**:

1. **Aislamiento de Red L3/L4:** Las *NetworkPolicies* y *Security Groups* prohíben que los pods de catálogo emitan paquetes hacia las subredes de datos; los intentos de conexión hacia el puerto 5432 de PostgreSQL se descartan en la interfaz virtual.
2. **Autenticación Mutua y Autorización Istio (L4/L7):** Todo intento de invocar APIs internas (como el servicio de Usuarios) requiere mTLS. El sidecar Envoy receptor comprueba la identidad SPIFFE del emisor (`spiffe://remarket.internal/ns/prod/sa/catalogo`) y la rechaza (HTTP 403) mediante su *AuthorizationPolicy*.
3. **Segregación de Permisos IAM y de BD:** El rol del catálogo (IRSA) no puede leer secretos de Secrets Manager ni usar las claves KMS de usuarios o pagos, y no tiene usuario en ninguna base PostgreSQL. Aunque llegara a la base, los campos sensibles están cifrados con claves que no puede usar.

14. Evaluación y Gestión de Riesgos

Nivel de riesgo = Impacto (1–3) × Probabilidad (1–3). Rangos: **6–9 Alto**, **3–4 Medio**, **1–2 Bajo**. La matriz completa, con riesgo residual, está en el Entregable 3.

ID	Categoría	Activo	Amenaza	Vulnerabilidad	I	P	Nivel
R1	Credenciales	Claves de AWS y Stripe	Robo de secretos	Secretos en código o en el repositorio	3	2	**6 Alto**
R2	Comunicaciones	Tráfico entre pods	Espionaje o suplantación este-oeste	Red interna sin autenticación mutua	3	1	**3 Medio**
R3	Almacenamiento	Respaldos de BD	Robo o borrado de respaldos	Snapshots sin cifrar o en la misma cuenta	3	2	**6 Alto**
R4	Disponibilidad	Tienda en línea	DDoS en temporada alta	Capacidad fija, sin filtrado en el borde	3	3	**9 Alto**
R5	Cadena de suministro	Imágenes en producción	Dependencia o imagen maliciosa	Sin SCA ni verificación de firma	3	2	**6 Alto**
R6	Pagos	Confirmación de cobros	Webhook falso de “pago aprobado”	Webhook sin verificación de firma	3	2	**6 Alto**
R7	Terceros	Avisos por SMS	Caída del proveedor de SMS	Un solo proveedor	1	2	**2 Bajo**

Tratamiento de los Riesgos Prioritarios

Riesgo	Tipo de tratamiento	Preventivo	Detectivo	Correctivo
R1	Mitigación	Gitleaks en pre-commit y CI, OIDC sin claves fijas	Secret scanning de GitHub y alarmas de CloudTrail	Revocar y rotar en Secrets Manager; revisar CloudTrail

Riesgo	Tipo de tratamiento	Preventivo	Detectivo	Correctivo
R4	Mitigación + transferencia	CloudFront, Shield Advanced y WAF (500 req/min por IP)	Alarmas de CloudWatch por latencia y errores 5xx	Autoescalado HPA/Karpenter y *runbook* de respuesta; el costo del escalado por DDoS lo cubre la protección de costos de Shield Advanced (transferencia)
R6	Mitigación + transferencia	HMAC-SHA256 en tiempo constante, ventana de 300 s y `event.id` único	Alerta en SIEM por firmas inválidas	Bloqueo en WAF y conciliación con Stripe; el manejo de tarjetas se transfiere a Stripe (PCI SAQ-A)
R7	Aceptación	—	Alarma de errores del proveedor	El correo sigue funcionando; un retraso de SMS no detiene ventas

15. Disponibilidad y Continuidad del Negocio

Para resistir la caída de un centro de datos y absorber picos de ventas (Hot Sale, Buen Fin), ReMarket combina redundancia, balanceo y autoescalado:

1. **Balanceo de carga:** CloudFront reparte el tráfico global y el ALB lo distribuye entre pods sanos de tres zonas; los *health checks* sacan de rotación a los que fallan.
2. **Múltiples zonas y redundancia:** EKS corre en 3 zonas de disponibilidad con al menos dos réplicas por servicio. RDS opera en Multi-AZ con réplica síncrona y *failover* automático (alrededor de 60 s).
3. **Autoescalado:** HPA ajusta réplicas por CPU y por longitud de las colas SQS; Karpenter agrega nodos EC2 cuando faltan.
4. **Respaldo, replicación y recuperación:** *Point-in-Time Recovery* en RDS y DynamoDB con copias en otra cuenta (sección 6), y *runbooks* de restauración probados cada trimestre.

Objetivos de Recuperación (RTO y RPO)

Servicio	RTO	RPO	Justificación
1. Pagos	**≤ 5 min**	**0**	Perder un cobro es perder dinero o cobrar dos veces. La réplica síncrona Multi-AZ no pierde transacciones si cae una zona; 5 min cubren el *failover* de la BD. Ante corrupción lógica (que la réplica también copia) se restaura con PITR y se concilia con Stripe, que es la fuente de verdad del cobro.
2. Auth y Usuarios	**≤ 15 min**	**≤ 5 min**	Sin login no hay compras, pero las sesiones activas siguen funcionando con sus tokens. PITR de RDS permite volver a un punto de hace pocos minutos.
3. Catálogo	**≤ 30 min**	**≤ 1 h**	Mientras se recupera, CloudFront sigue sirviendo la última versión en caché; el catálogo cambia pocas veces al día, así que perder una hora de cambios es aceptable.

16. Uso de Inteligencia Artificial en Seguridad

Analizamos la IA desde dos lados: como herramienta del atacante y como evaluador de nuestro diseño. El detalle completo está en el Entregable 5.

16.1 La IA como Atacante

Escenario	Activo	Debilidad	Impacto	Controles de ReMarket	Evidencia para detectarlo
Phishing dirigido a soporte (la IA redacta correos personalizados a escala)	Cuentas del personal y consola AWS	Factor humano	Acceso a datos de clientes y a la nube	MFA FIDO2 resistente a phishing, VPN y roles temporales	Inicios de sesión desde IP o país inusual (GuardDuty, CloudTrail)

Escenario	Activo	Debilidad	Impacto	Controles de ReMarket	Evidencia para detectarlo
Búsqueda automática de fallas BOLA (la IA infiere la API y genera peticiones válidas)	Pedidos y perfiles de clientes	Autorización por objeto mal implementada	Fuga masiva de pedidos y direcciones	Cliente tomado del `sub` del JWT, validación OpenAPI, rate limiting	Ráfagas de 403/404 con IDs secuenciales desde una sesión (SIEM)
Búsqueda de secretos filtrados (la IA detecta claves ofuscadas que escapan a regex)	Llaves de Stripe y AWS	Secretos en código o historial de Git	Cobros fraudulentos y control de la nube	Gitleaks en pre-commit y CI, OIDC, Secrets Manager	Alertas de *secret scanning* y uso de credenciales desde IPs nuevas (CloudTrail)

16.2 La IA como Evaluador

Le pedimos a una IA (Claude, de Anthropic) que auditara un resumen de la arquitectura. Prompt (resumido): *"Actúa como un Auditor Senior de Ciberseguridad Cloud... Evalúa el siguiente diseño... Identifica al menos tres observaciones técnicas"*. Prompt y respuesta completos en `entregables/5-analisis-ia/evidencia/`.

Obs.	Fragmento de la respuesta	¿Correcta? ¿Aplica?	Riesgo que reduce	Decisión y evidencia
1	*"Si un servicio con permiso de publicación es comprometido, puede inyectar un `PedidoPagado` falso..."*	Sí y sí: SSE-KMS no prueba el origen del evento	Eventos falsos, despachos sin cobro	**Modificada:** firmamos sólo `PedidoPagado` con KMS en vez de separar temas; Kafka descartado (sec. 11)
2	*"Dentro de esos 5 minutos el mismo evento firmado puede llegar varias veces... Registren el `event.id`"*	Sí y sí: la ventana limita *cuándo*, no *cuántas veces*	Pagos procesados dos veces	**Aceptada:** registro de `event.id` y verificación de monto con Stripe (sec. 9)
3	*"Si se guardan en `localStorage`, cualquier XSS puede leerlos..."*	Sí, y exhibió una inconsistencia nuestra (cookie vs. Bearer)	Robo de sesión por XSS	**Modificada:** acceso en memoria y refresco en cookie `HttpOnly` (secs. 2 y 5)

16.3 Valoración Crítica

Las tres observaciones fueron correctas, pero dos se ajustaron a nuestro sistema antes de aplicarlas, y la IA no vio errores que no estaban en el texto que le dimos (el webhook dibujado directo a un pod privado).

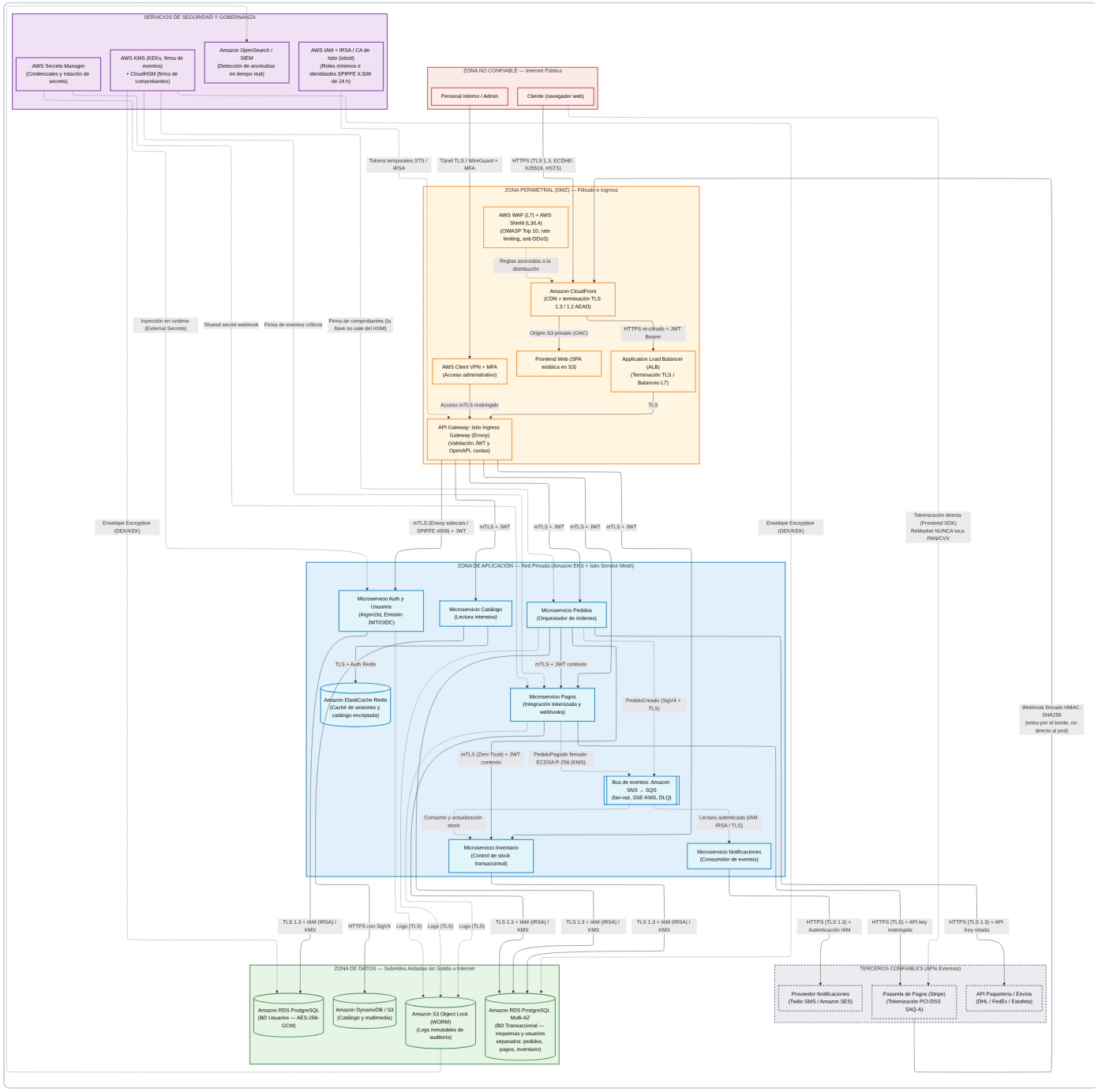
17. Reflexión Final: Límites de la Inteligencia Artificial en Ciberseguridad

Los LLMs ayudan a revisar diseños y proponer defensas, pero **una organización no debe aplicar automáticamente todo lo que recomienda una IA**:

1. **Contexto:** La IA sólo conoce lo que se le muestra; una recomendación correcta en general puede no aplicar al sistema concreto.
2. **Falsos positivos:** Los modelos marcan como vulnerables cosas que no lo son (una dependencia con una CVE en una función que nunca se llama, un "secreto" que es un ID público). Atenderlos sin verificar introduce cambios innecesarios y acostumbra al equipo a ignorar alertas.
3. **Alucinaciones:** Pueden inventar parámetros o configuraciones inexistentes con total convicción.
4. **Configuración real del sistema:** Una sugerencia pensada para un servidor tradicional puede ser imposible en un servicio gestionado (por ejemplo, mTLS en SQS) o romper la integración con Istio.
5. **Impacto operativo:** Reglas de WAF demasiado estrictas o firmas en cada lectura del catálogo pueden bloquear compras legítimas o tirar la tienda en temporada alta.
6. **Gestión de riesgos:** No existe la seguridad al 100%. Decidir qué riesgo se mitiga, transfiere o acepta depende del costo y del negocio, como hicimos en la sección 14; un modelo no tiene esa información.
7. **Responsabilidad humana:** PCI-DSS, la LFPDPPP o el GDPR responsabilizan a la empresa y a sus personas. Ningún auditor acepta "lo recomendó una IA" como justificación.

La IA es un **copiloto**: propone y acelera la revisión. La decisión final, su contexto y su responsabilidad son del equipo.

Anexo A. Diagrama de Arquitectura



Arquitectura cloud de ReMarket: zonas de confianza, servicios, bases de datos, comunicaciones y componentes de seguridad. Fuente editable: entregables/1-diagrama-arquitectura/arquitectura.mmd